Forgotten Post-Mortem

Forgotten

Team Isaac

Isaac Mayernik

8 December 2024

Introduction

Forgotten was developed in Unreal Engine 5 using the blueprints system to develop gameplay systems. It is a 3D action game with soul-like mechanics such as stamina and dodging. The game was intended to have an RPG element, such as an XP system, like in Dark Souls, but there was not enough time. The game has slight differences from its original proposal and game design document.

Features Implemented

The player interacts with the game in many ways, whether by pressing keys or game elements. This includes using the menus, attacking, interacting with the rest point, dodging, healing, pausing, and moving. Some limitations include movement restrictions like while at a rest point, healing, or attacking, combat restrictions such as stamina, and healing restrictions limited by the number of healing items left in the player's inventory. While there is no win screen, by defeating the golem boss and reaching the next rest point, the player has won. By losing all their health, the player dies and loses but will be respawned so the player may try again. To reach the goal of the rest point after the boss, the player must defeat bunny enemies along the path. These enemies will attack the player with their kunai. The golem boss is the final obstacle before the last rest point. The boss will swipe its arms and slam them on the ground to attack the player. The player must avoid these attacks and defeat the boss.

Game Systems

1. Respawn system

When the player dies, several events occur: inputs are disabled, the game over widget is displayed, and the game checks if the respawn location is set to the default or a rest point. If true, a new actor is spawned at the default position. Health, stamina, and healing items are reset, and enemies are respawned. If the player had previously picked up the sword, the sword mesh is re-equipped.

2. Enemy reset system

The game tracks the enemies' positions and resets them when the player dies or interacts with a rest point. All enemy data is stored in a struct array to ensure enemies are respawned at their default positions without duplicating actors.

3. Player attack system

The player has a combo attack system. When the player clicks the left mouse button, the game checks if the weapon is equipped, the player's stamina, and if the player is already attacking. If all conditions are met, the attack animation plays. A damage sphere trace is performed, and damage is dealt to enemy actors within the trace.

4. Rest point system

The rest point system allows the player to rest, heal, and reset enemies. A tooltip appears when the player is in range of a rest point, and pressing a button lets them interact with it. While resting, the player can't move, and all enemies are reset.

5. Stats system

This system tracks the health and stamina of the player and enemies. The health bar is updated based on damage dealt, and stamina is consumed when performing actions like dodging and attacking.

6. Dodge system

When the player presses the spacebar, the dodge animation plays, provided the player is not already dodging and has enough stamina. The player regains stamina after a second if they do not attack or dodge again.

7. Heal system

The player can heal by pressing the R key, provided healing items are available. If the player has no items, a "heal empty" animation plays. Health is restored and the inventory is updated accordingly.

8. Weapon pickup system

This system allows for multiple weapons to be picked up and the mesh to be updated based on the data asset of the weapon. However, in the final build, the player may only pick up a sword. When the player overlaps the collision of the sword, it gets the data, updates the mesh and attaches it to the socket based on the data, and then updates the icon in the inventory HUD based on the data.

9. Save system

The save holds variables such as whether the weapon was equipped by the player, the weapon data, the player's heal count, if the golem boss had been defeated, and the last position of the player. When the player starts the game from the main menu, there will be no save file so one will be created, and all these variables will be set to their default. When the player quits, all these variables will be checked and compared to the ones in the game instance. The ones in the game instances are always up to date. Therefore, the save file variables are updated and so is the save file. When the player starts the game the next time, everything will be updated in the game instance based on the save variables, and so if the player had the weapon equipped when they saved, the weapon will be re-equipped when starting the game again.

10. Music system

The music system dynamically changes based on game events. When entering the boss arena, the music transitions from the background theme to the boss theme. The theme changes again when the boss is defeated or if the player dies.

Visual and Audio Assets

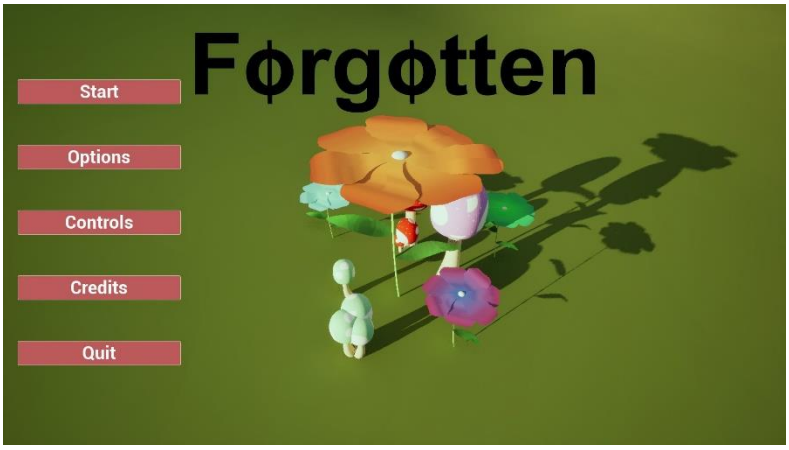


Figure 1: an image of the main menu. There are buttons to start and quit the game, as well as travel to the other menus.

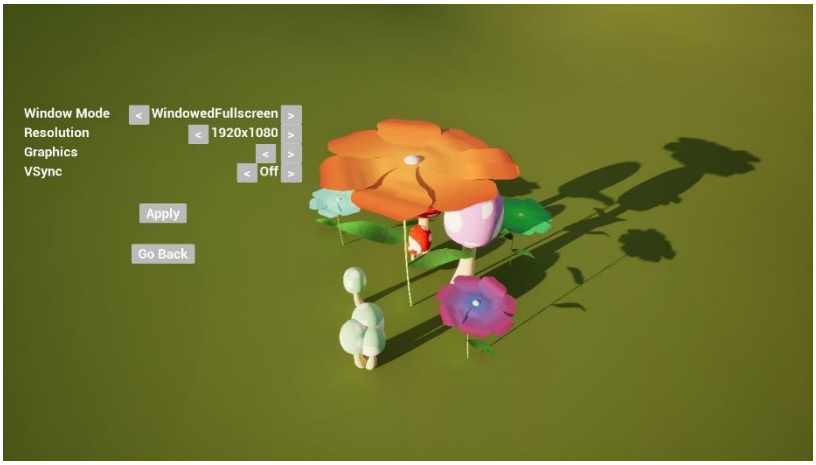


Figure 2: an image of the options menu. There are buttons to change each setting from a list of preset options. These options, once applied, are saved for the user when revisiting the menu or quitting.

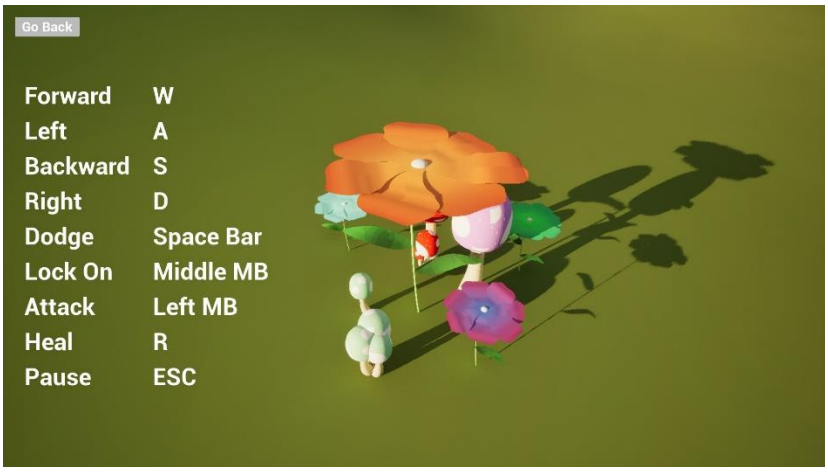


Figure 3: an image of the controls menu.

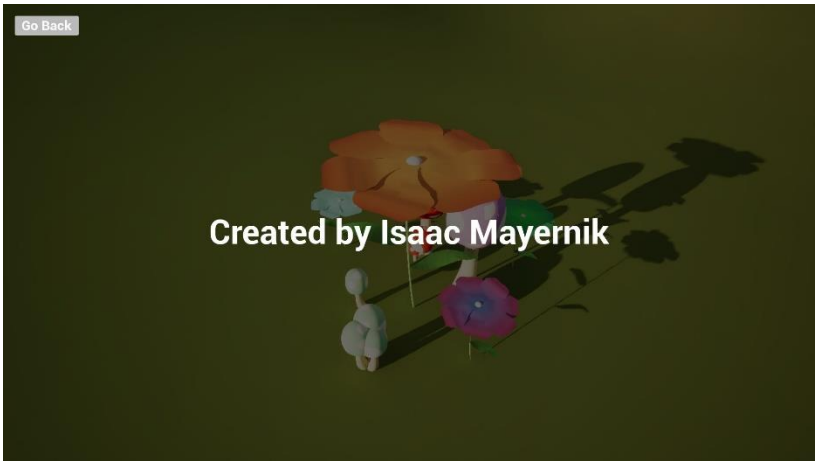
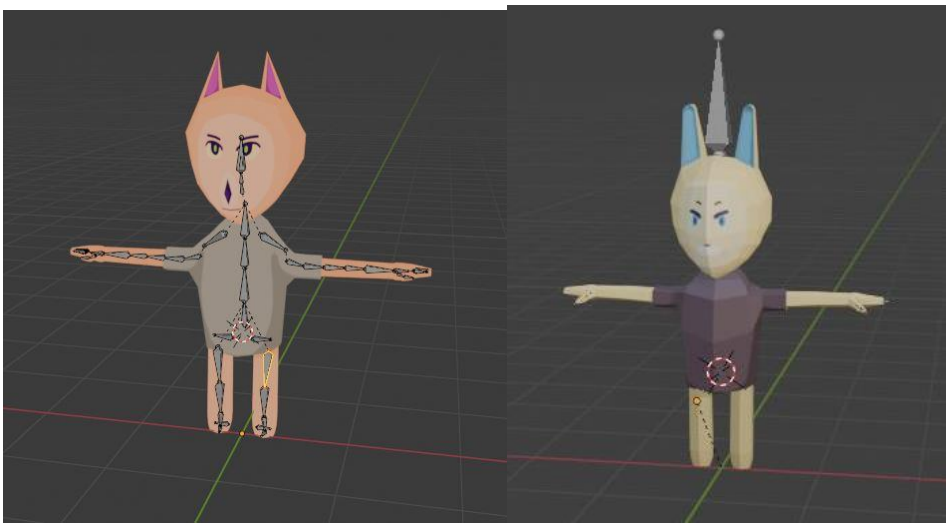


Figure 4: an image of the credits menu.

Figure 5: an image of the bonfire menu. There is a button that closes the menu. The player is unable to move or do any input while in this menu.

Figure 6: an image of the pause menu. The game does not actually pause, meaning enemies may still attack the player. From top to bottom, there are buttons to close this menu, save the game and go to the main menu, and save the game and quit.



Figures 7 and 8: images of the player fox model and enemy bunny model.

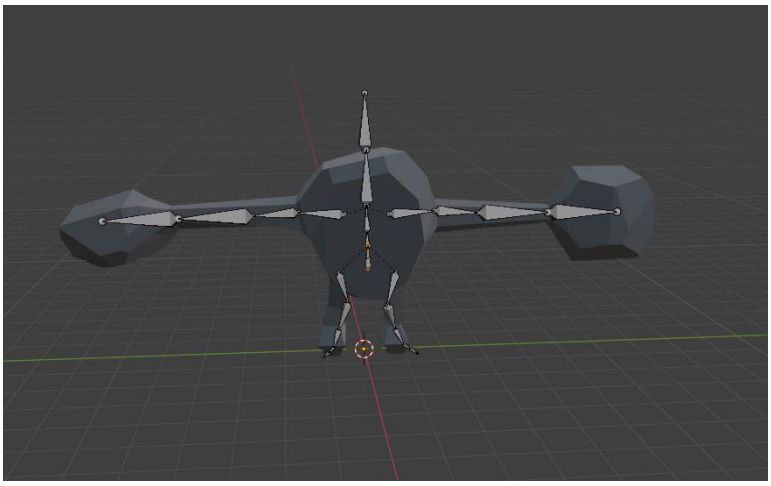


Figure 9: an image of the golem boss model.

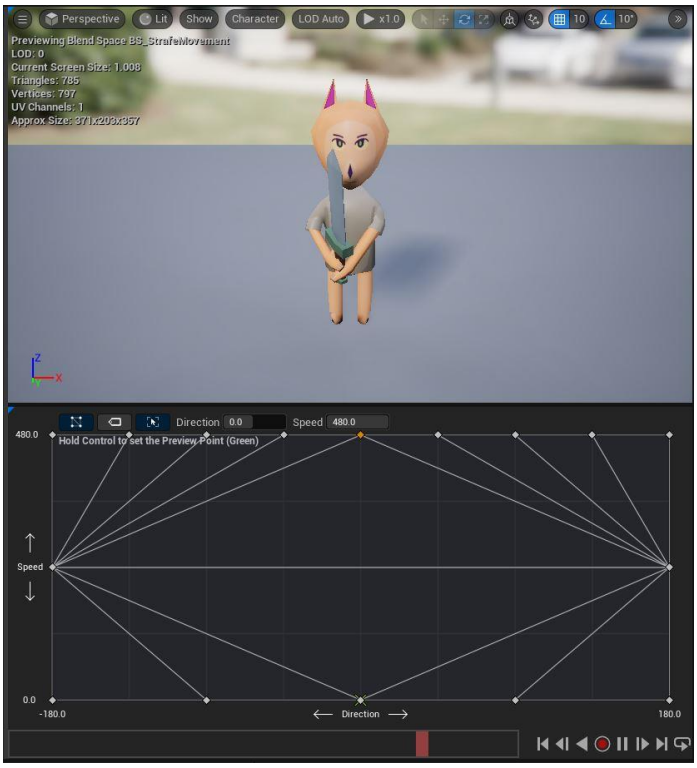


Figure 10: an image of the player fox movement blendspace to be used in the animation graph.

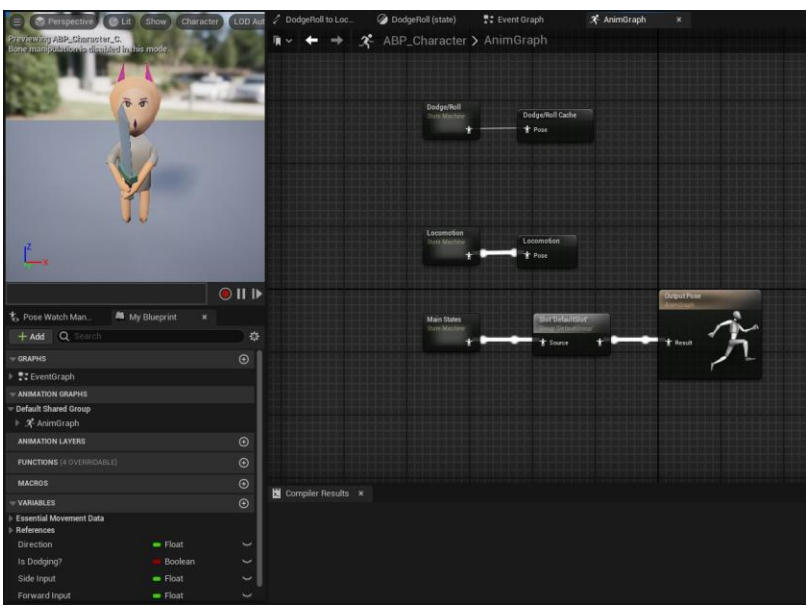
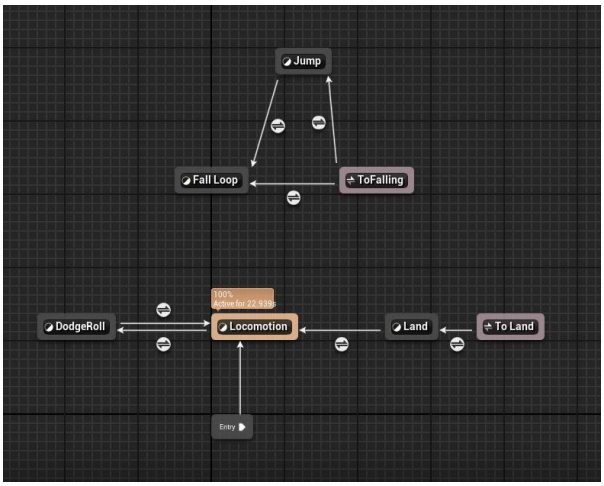
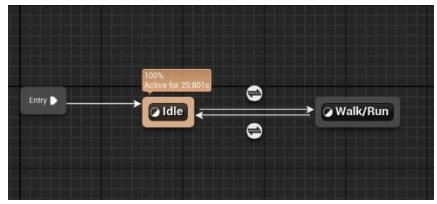
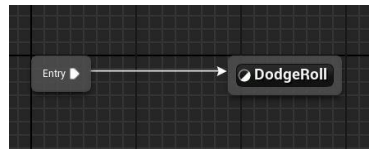


Figure 11: an image of the player animation graph.







Figures 12, 13, and 14: images showcasing the various states of the player animation graph. First is the main state that controls the states. It automatically starts at locomotion, which will start at the idle animation. If should move is true, which is if there is directional input, it will transition to walk/run, which will use the movement blendspace. From locomotion, it can transition to dodge roll if the player is dodging which occurs when the space bar is pressed. If true, the dodge animation will happen. While there is no jump key, the fall transitions seen at the top make sure the player stays on the ground and in the idle animation.

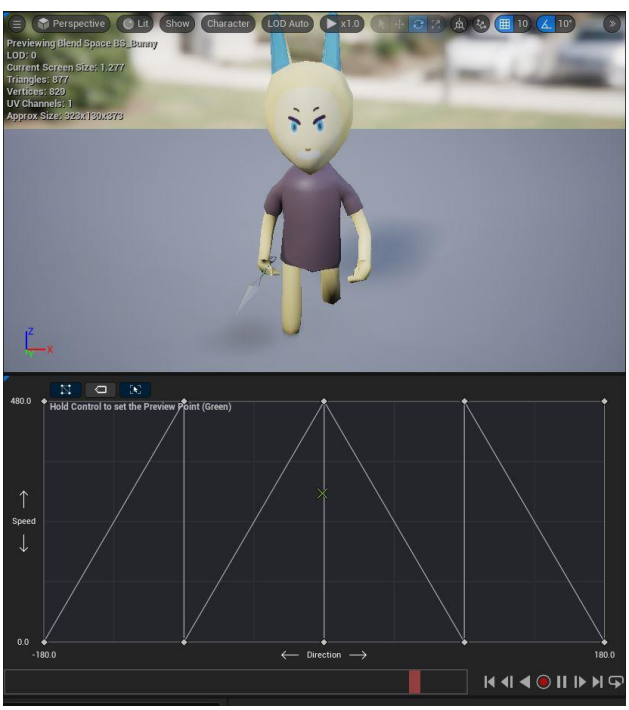


Figure 15: an image of the enemy bunny movement blendspace to be used in the animation graph.

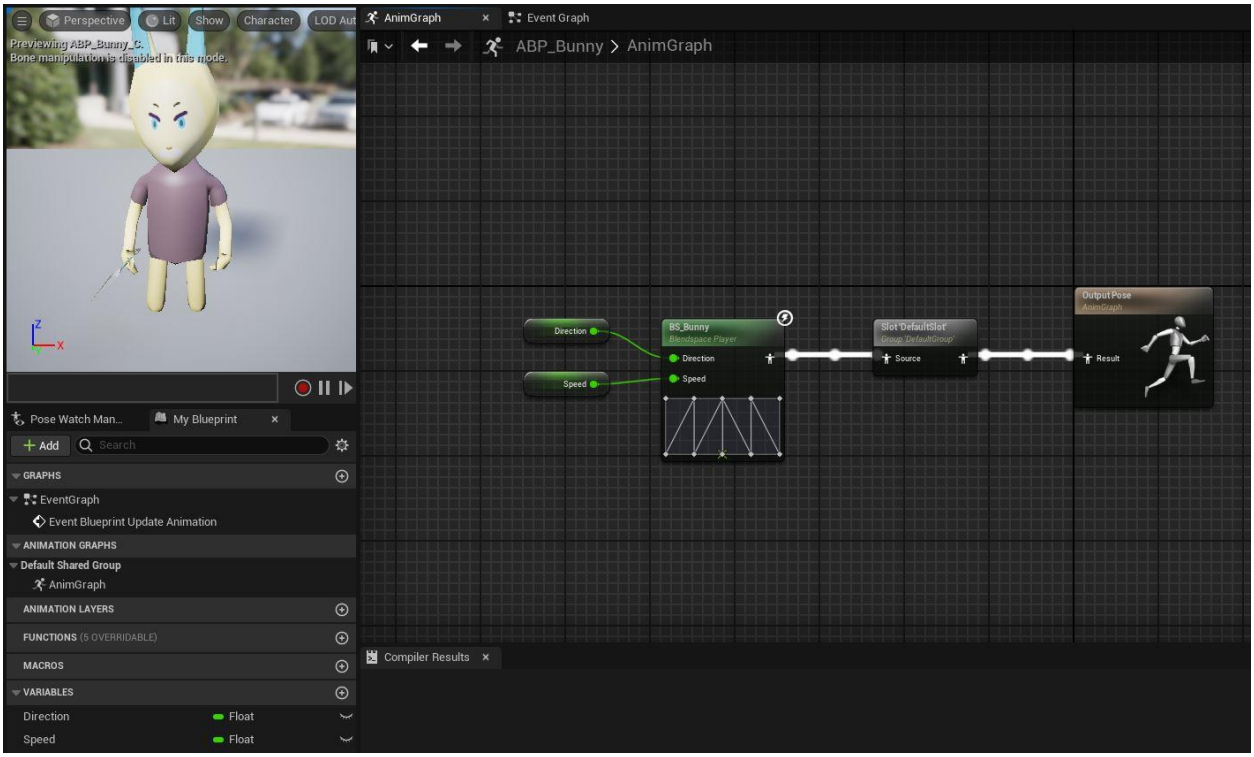
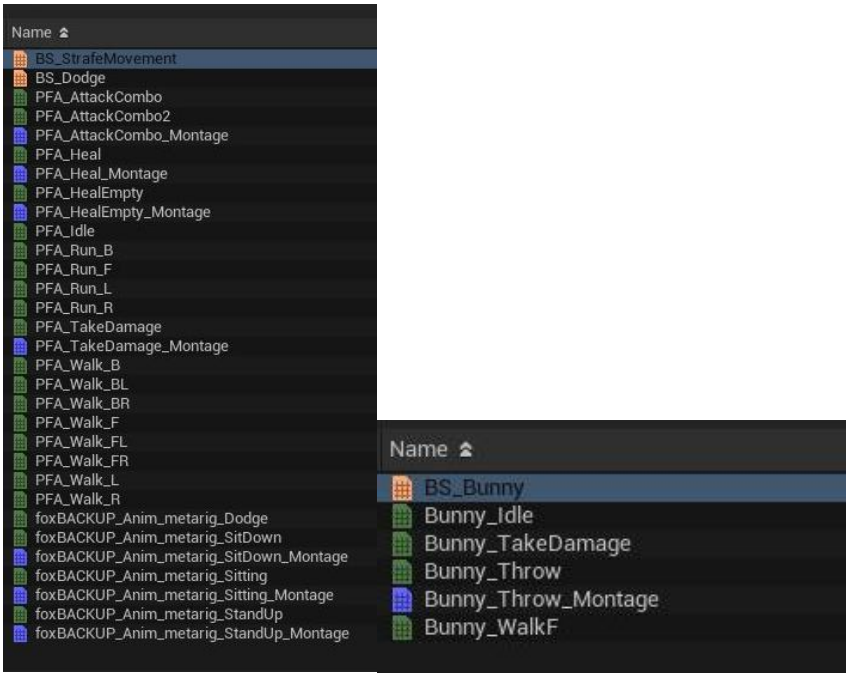


Figure 16: an image of the enemy bunny animation graph. If the player is in sight of the enemy, it will play the walking animation as it walks forward.



Figures 17 and 18: lists of the player fox and enemy bunny animations.

There are three themes found in the final build of Forgotten:

1. Main theme

The main theme can be heard playing while on the main menu. While it may not fit the theme of Forgotten, I wanted to create something like the main theme of *Dark Souls 1* but with choir notes. I listened to it a few times, and while it sounded like a piano, I realized it was a cello and violin. With that in mind, I made some notes that I thought sounded good and dramatic and added some choir and harp notes to elevate it. I tried adding a light piano sound in the background but did not think it fit well with the rest of the instruments.

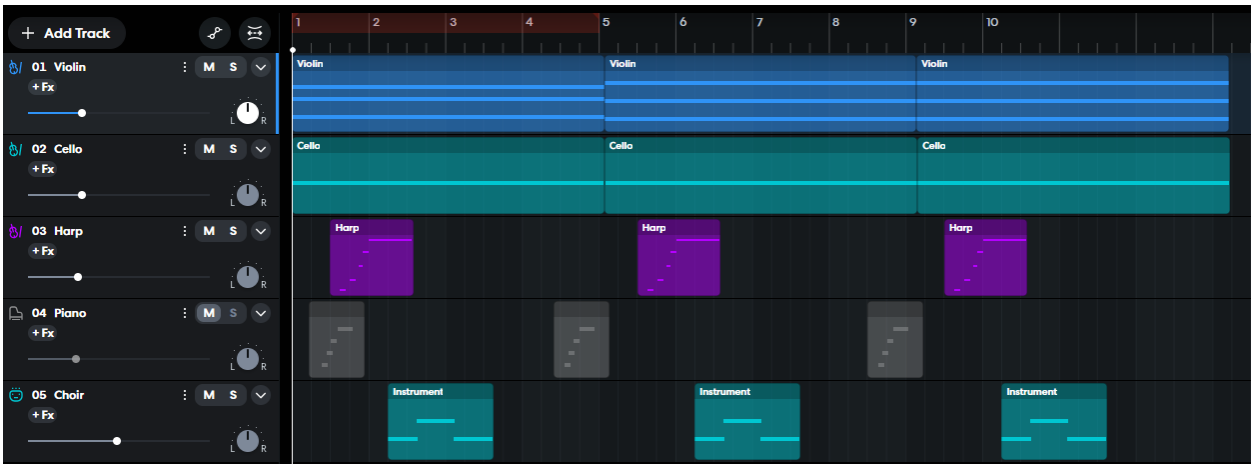


Figure 19: image of the Bandlab track showing the instruments used in the title theme.

2. Background theme

For the background theme, I wanted to create something calm that would fit a forest. Bandlab has pads as an instrument, and so I found the atmosphere pad which I thought fit the tone I wanted. I also added a chorus organ to play halfway through to add a bit of dramatic noise as the player will be fighting enemies in the forest.

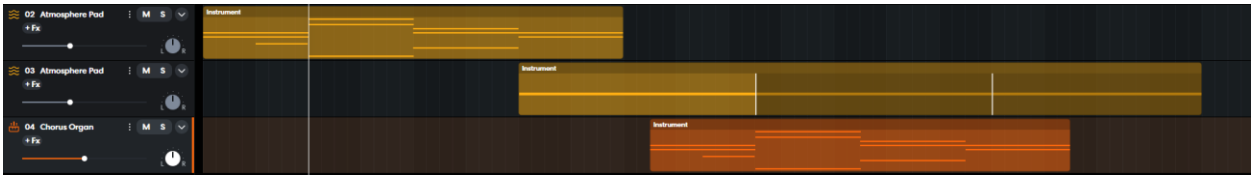


Figure 20: image of the Bandlab track showing the instruments used in the background theme.

3. Golem boss theme

For the golem boss theme, I wanted something dramatic that would get the player's heart racing. I added a synth that plays fast and lightly in the background, while the grand piano strikes a deep chord to add dramatic noise. While basic, I think the theme gets the job done.

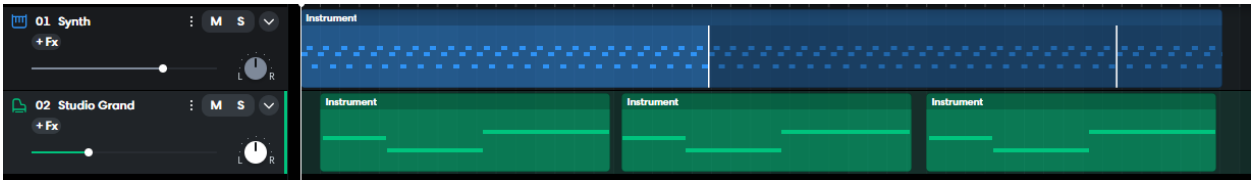


Figure 21: image of the Bandlab track showing the instruments used in the golem boss theme.

All sound effects were created using a simple phone or headset microphone. Here is a list of the sound effects and how they were created:

- Player hurt sound effect – saying "grah" and speeding up
- Enemy hurt sound effect – some sound from my mouth, I do not remember how
- Dodge/bonfire use sound effects – blowing air out lightly
- UI click sound effect – clicking the roof of my mouth with my tongue
- Slash sound effect – slashing a lightly filled water bottle through the air
- Slam sound effect – dropping a lightly filled water bottle on wood
- Heal sound effect – crunching with my mouth

Features Removed

Here are some features from the proposal that had to be deprecated:

1. Bunny boss and second level

The proposal details the bunny boss being at the end of the second area. Both did not get started due to time constraints and prioritization of other features.

2. Beetle enemy

While the model was created, I prioritized gameplay systems instead of creating its animations, resulting in the enemy not being added to the game. Implementing the enemy did not happen due to time constraints.

3. XP system

By defeating an enemy, the player would earn a set amount of experience points to be used to level up a stat at the rest point. The player could see their current XP at the bottom-right of the screen. However, the system was not created due to time constraints. I believe implementing it would not have been difficult and could have been added if more time were available.

4. Roll animations

While implemented in the proof of concept, this feature had to be removed due to incorrectly importing the next removed feature into Unreal Engine. Although the feature was implemented, the root of the player was not moving in the engine, causing the player to 'teleport' back to their original position. Overall, the roll animations and player rig not being redone are due to time constraints.

5. Idle sitting animation

While importing this animation into Unreal Engine, I misread some import settings and broke the player character's skeleton. This broke many animations, including this one from working. Ultimately it had to get removed and was not fixed due to time constraints.

6. Bunny enemy throwing kunai

Upon creating the system, I had trouble getting the kunai to be thrown from the correct position of the bunny's hand. When it did, it would not damage the player, would fall indefinitely, and would not destroy. Ultimately, I circumvented this issue by having the bunny enemy hold the kunai and swing it at the player.

These are features I wanted to add but were not part of the proposal:

1. NPC with dialogue

The world in the final build feels empty, with only enemies populating the level. Adding an NPC with dialogue would have given some interaction and a more immersive experience. However, this feature was not implemented due to time constraints.

2. Slash effect

I felt adding this would make the sword attacks more engaging. Unfortunately, I was unable to figure out how to implement this effect so that it would trigger every time the player attacked without looping. Due to this, the feature was ultimately scrapped.

3. Player-modified controls and menu

By adding this there would be customizability to the controls so players wouldn't have to use the ones I preset. Unfortunately, Unreal Engine changed how the input actions work in version 5, and I could not figure out and finish this.

Problems Encountered

1. Roll animations

A major issue I encountered was setting up the player's roll animations. Despite moving the root bone of the player when creating the animation, Unreal Engine, even with root motion enabled, did

not move the player in the intended direction without returning them to their original position. The only solution would have been to redo the entire rig and all the animations, but I did not have time, so the issue was unresolved.

2. Sitting animation

Another problem arose when creating the player's idle sitting animation. Blender would often not save my animations properly, so I had to recreate the player's sitting animation. After importing it into Unreal Engine, I misread some settings and broke the player skeleton, causing all its animations to break. To 'fix' this, I had to remove the roll dodges and idle sitting animations from the final build. Unfortunately, this issue would again require redoing all the animations Blender failed to save.

3. Boss enemy damage issue

A third major issue was the boss enemy dealing damage to itself instead of the player. By debugging, I discovered the game was dealing damage to the player but displayed the damage on the boss's health bar instead of the player's as the health bar reference was replaced by the boss's. The solution was to implement another method for handling damage to the boss and its health bar, allowing both health bars to display their health.

4. Respawning player and sword mesh

I created a respawn system that worked, but the sword, a pickup item, had its static mesh reset to nothing when the new character spawned in. It took me about three weeks to resolve this issue using a cast-to-character node to update the sword mesh correctly. The game had attempted to update the discarded character's sword mesh, which caused the problem.

Conclusion

Developing a 3D game was not that bad, and I would do it again, but not with Unreal Engine. For one, Epic Games is pushing Unreal Engine to create games with realistic graphics by pushing features like raytracing and Nanite support, which I have no interest in. Another issue I encountered was unnecessarily large file sizes for builds, evident in both this project and my high school senior project. Additionally, while the blueprint system has its advantages, I preferred coding with Unity. When working in Unity on my 2D game, I could more easily identify and resolve issues within the code. As this project grew, especially with the character blueprint, the increasing complexity and difficulty of debugging was apparent. This was particularly frustrating when saving variables in the save file, something I found much easier to do in Unity. The great challenge during development was finding time to work on the game. The first two or three weeks were fine until I had exams in my other courses which caused me to put this aside for a while. Not to mention the capstone project coding process began around the same time as this project, meaning I had to find time to code my fair share of the capstone with everything else. Overall, I am very proud of the work accomplished, and I will likely continue working on this in the next few months to finish everything I did not get done when I can find time.